Gérer les noms d'hôte personnalisés

Les associations entre les IP et les noms d'hôte sont faites sur ce que l'on appelle des *serveurs DNS*. Nous n'allons pas entrer dans le détail, mais sachez en gros que chaque fournisseur d'accès met en place des serveurs DNS qui fournissent la liste des correspondances IP ↔ noms d'hôte.
Si vous voulez en découvrir plus sur le fonctionnement des DNS, je vous invite à lire mon tutoriel sur les DNS.

Ainsi, lorsque vous tapez `siteduzero.com` dans votre navigateur, vous pouvez obtenir l'adresse IP correspondante et naviguer sur le Site du Zéro.
C'est quand même plus pratique que d'avoir à retenir l'IP !

Vous ne pouvez pas modifier la liste des correspondances IP ↔ noms d'hôte sur le serveur DNS (puisque ce serveur est utilisé par de nombreuses personnes), mais en revanche vous pouvez établir une liste de correspondances personnalisée sur votre ordinateur.

Ouvrez pour cela en root le fichier `/etc/hosts` :

```
$ sudo nano /etc/hosts
```

Dedans, vous devriez avoir des lignes ressemblant à ceci :

```
127.0.0.1       localhost
127.0.1.1       mateo21-laptop
```

À gauche l'IP, à droite le nom d'hôte correspondant. Écrire `localhost` est donc équivalent à écrire `127.0.0.1` .

Vous pouvez ajouter des lignes sur le même modèle pour faire correspondre une IP à un nom d'hôte.

> Quel intérêt ? Cela dépend. Parfois, les DNS ne fonctionnent pas bien pendant de courtes périodes (c'est très rare, mais ça peut arriver). Dans ce cas, il est plus simple de modifier votre fichier `hosts` pour pouvoir continuer à consulter votre site préféré en « forçant » l'association du nom d'hôte et de l'IP.
>
> Vous pourriez donc ajouter :
>
> ```
> 92.243.25.239      siteduzero.com
> ```
>
> Enregistrez, ouvrez un navigateur, puis tapez `siteduzero.com` pour voir si ça fonctionne.

> Attention : cette technique a l'avantage de forcer l'association, mais si notre serveur change un jour d'IP, votre ordinateur ne sera pas au courant ! En règle générale, il est préférable d'utiliser les serveurs DNS qui se mettent régulièrement à jour (une fois par jour, en moyenne) afin d'avoir toujours une liste actualisée.

Sur un réseau local, il peut être pratique d'associer un nom d'hôte à chaque PC pour pouvoir vous y connecter sans avoir à retenir l'IP :

```
192.168.0.5      pc-papa
```

Ainsi, écrire `pc-papa` vous permet d'accéder à cet ordinateur sans avoir à retenir l'adresse IP correspondante.

ifconfig & netstat : gérer et analyser le trafic réseau

Nous allons découvrir ici deux commandes : `ifconfig` et `netstat` . La première permet de gérer les connexions réseau de votre machine (pour les activer / désactiver, par exemple) tandis que la seconde vous permet d'analyser ces connexions, de connaître des statistiques, etc.

`ifconfig` : liste des interfaces réseau

Votre ordinateur possède en général plusieurs **interfaces réseau**, c'est-à-dire plusieurs moyens de se connecter au réseau.

Tapez `ifconfig` dans la console pour voir ce que ça donne :

```
$ ifconfig
eth0      Link encap:Ethernet  HWaddr 00:90:f5:56:44:5a
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          Packets reçus:0 erreurs:0 :0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 lg file transmission:1000
          Octets reçus:0 (0.0 B) Octets transmis:0 (0.0 B)
          Interruption:220 Adresse de base:0xe000

lo        Link encap:Boucle locale
          inet adr:127.0.0.1  Masque:255.0.0.0
          adr inet6: ::1/128 Scope:Hôte
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          Packets reçus:10 erreurs:0 :0 overruns:0 frame:0
          TX packets:10 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 lg file transmission:0
          Octets reçus:500 (500.0 B) Octets transmis:500 (500.0 B)

wlan0     Link encap:Ethernet  HWaddr 00:19:d2:61:90:0a
          inet adr:192.168.1.2  Bcast:192.168.1.255  Masque:255.255.255.0
          adr inet6: fe80::219:d2ff:fe61:900a/64 Scope:Lien
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          Packets reçus:5238 erreurs:0 :0 overruns:0 frame:0
          TX packets:4899 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 lg file transmission:1000
          Octets reçus:5069449 (5.0 MB) Octets transmis:1202459 (1.2 MB)
```

Configurez le nom du réseau de votre serveur

Premier élément de configuration réseau du serveur : **son nom réseau**.

> *i* Le nom réseau d'un serveur Linux (aussi appelé "**system hostname**") sert d'identifiant par défaut de la machine pour tous les services et applications qui s'exécutent en local et pour beaucoup de services et d'applications lorsque ceux-ci communiquent sur le réseau.

Prenez l'exemple d'un serveur de fichiers logs centralisés sur un réseau interne : toutes les communications logs des serveurs du réseau seront préfixées de leur nom réseau, ce qui permettra au service de centralisation des logs de savoir d'où viennent tels ou tels fichiers.

Si tous les serveurs de logs ont le même nom réseau (par exemple **`localhost`** qui est proposé pendant le processus d'installation), difficile de reconnaître l'origine des fichiers communiqués.

Dans la vidéo ci-dessous, je vous présente :

1. le fichier **`/proc/sys/kernel/hostname`** qui contient la valeur courante du nom réseau du serveur,
2. la commande **`hostnamectl`** qui permet de la modifier,
3. le fichier **`/etc/hostname`** qui permet d'initialiser le nom réseau au démarrage.

```
root@debServer:~# cat /proc/sys/kernel/hostname
debServer
root@debServer:~# sysctl kernel.hostname
kernel.hostname = debServer
```

seb@debServer: ~

```
root@debServer:~# hostnamectl status
   Static hostname: debServer
         Icon name: computer-vm
           Chassis: vm
        Machine ID: f4347f1a943f42879acab0a44c8a0600
           Boot ID: a8043cc19a114eb5a443e0d7f6534ada
    Virtualization: oracle
  Operating System: Debian GNU/Linux 11 (bullseye)
            Kernel: Linux 5.10.0-8-amd64
      Architecture: x86-64
root@debServer:~# hostnamectl set-hostname vega
root@debServer:~# hostnamectl status
   Static hostname: vega
         Icon name: computer-vm
           Chassis: vm
        Machine ID: f4347f1a943f42879acab0a44c8a0600
           Boot ID: a8043cc19a114eb5a443e0d7f6534ada
    Virtualization: oracle
  Operating System: Debian GNU/Linux 11 (bullseye)
            Kernel: Linux 5.10.0-8-amd64
      Architecture: x86-64
root@debServer:~# sysctl kernel.hostname
kernel.hostname = vega
root@debServer:~#
root@debServer:~# cat /etc/hostname
vega
root@debServer:~#
root@debServer:~#
root@debServer:~# bash
root@vega:~# exit
exit
root@debServer:~#
```

Détectez les interfaces réseaux de votre système



Avant toute configuration, il est nécessaire de s'assurer que le noyau Linux a bien détecté les cartes réseaux connectées d'une manière ou d'une autre sur la machine.

La commande **`dmesg`** (pour "display message") permet d'afficher les messages du noyau :

- pendant le processus de démarrage,
- et notamment lorsque ce dernier charge les pilotes des périphériques qui vérifient si un matériel connecté leur est compatible.

Voyons maintenant comment :

1. lancer la commande **`dmesg`** pour essayer de détecter les cartes réseaux reconnues par le noyau lors du démarrage,
2. consulter le système de fichier virtuel **`/sys`** pour lister les interfaces réseaux,
3. et au passage, de constater une évolution liée à **`Systemd`** dans le nommage par défaut de ces interfaces. On y va ?

```
root@debServer:~# dmesg | grep Network
[    2.024307] e1000: Intel(R) PRO/1000 Network Driver
[    2.731563] e1000 0000:00:03.0 eth0: Intel(R) PRO/1000 Network Connection
[    3.248558] e1000 0000:00:08.0 eth1: Intel(R) PRO/1000 Network Connection
root@debServer:~#
root@debServer:~# dmesg | grep eth0
[    2.731558] e1000 0000:00:03.0 eth0: (PCI:33MHz:32-bit) 08:00:27:46:df:1e
[    2.731563] e1000 0000:00:03.0 eth0: Intel(R) PRO/1000 Network Connection
[    3.260269] e1000 0000:00:03.0 enp0s3: renamed from eth0
root@debServer:~# dmesg | grep eth1
[    3.248554] e1000 0000:00:08.0 eth1: (PCI:33MHz:32-bit) 08:00:27:c6:ca:e0
[    3.248558] e1000 0000:00:08.0 eth1: Intel(R) PRO/1000 Network Connection
[    3.251875] e1000 0000:00:08.0 enp0s8: renamed from eth1
root@debServer:~#
root@debServer:~# ls -l /sys/class/net/
total 0
lrwxrwxrwx 1 root root 0 22 oct.  16:40 enp0s3 -> ../../devices/pci0000:00/0000:
00:03.0/net/enp0s3
lrwxrwxrwx 1 root root 0 22 oct.  16:40 enp0s8 -> ../../devices/pci0000:00/0000:
00:08.0/net/enp0s8
lrwxrwxrwx 1 root root 0 22 oct.  16:40 lo -> ../../devices/virtual/net/lo
root@debServer:~#
```

Configurez les cartes réseaux de manière dynamique

Pour les plus anciens d'entre nous, la commande permettant de configurer une interface réseau de manière dynamique était `ifconfig` .

> Cependant cette commande et le package associé `net-tools` ne sont plus maintenus depuis 2009. Notamment parce que la communauté de développeurs considérait qu'elles n'étaient pas optimisées, ne profitant pas non plus des nouvelles fonctionnalités des noyaux Linux. Ainsi la commande `ifconfig` mais également des commandes comme `netstat` , que j'ai beaucoup utilisées, ont disparu.

Heureusement pour nous, le package `iproute2` fournit les commandes qui remplacent les précédentes.

En l'occurrence, désormais, sur toutes les distributions Linux, vous pouvez configurer de manière dynamique les cartes réseaux avec la commande `ip` .

Voyons comment utiliser la commande `ip` pour :

1. gérer vos interfaces réseaux,
2. et notamment leur attribuer une adresse de manière dynamique.

```
root@debServer:~# ip link
add     delete  help    set     show
root@debServer:~# ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT
 group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
mode DEFAULT group default qlen 1000
    link/ether 08:00:27:46:df:1e brd ff:ff:ff:ff:ff:ff
3: enp0s8: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT gro
up default qlen 1000
    link/ether 08:00:27:c6:ca:e0 brd ff:ff:ff:ff:ff:ff
root@debServer:~# ip link set enp0s8 down
root@debServer:~# ip link set enp0s8 up
root@debServer:~#
root@debServer:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
group default qlen 1000
    link/ether 08:00:27:46:df:1e brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.24/24 brd 192.168.1.255 scope global dynamic enp0s3
       valid_lft 85543sec preferred_lft 85543sec
    inet6 2a01:cb00:85af:f600:a00:27ff:fe46:df1e/64 scope global dynamic mngtmpa
ddr
       valid_lft 1753sec preferred_lft 553sec
    inet6 fe80::a00:27ff:fe46:df1e/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
group default qlen 1000
    link/ether 08:00:27:c6:ca:e0 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::a00:27ff:fec6:cae0/64 scope link
       valid_lft forever preferred_lft forever
root@debServer:~#
```

Ajouter une adresse IP à enp0s8

```
root@debServer:~# ip addr add 192.128.1.24/24 dev enp0s8
root@debServer:~#
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
group default qlen 1000
    link/ether 08:00:27:c6:ca:e0 brd ff:ff:ff:ff:ff:ff
    inet 192.128.1.24/24 scope global enp0s8
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fec6:cae0/64 scope link
       valid_lft forever preferred_lft forever
```

Retirer l'adresse IP de l'interface enp0s8

```
root@debServer:~#
root@debServer:~# ip addr del 192.128.1.24/24 dev enp0s8
```

Afficher l'interface enp0s8

```
root@debServer:~# ip addr show enp0s8
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
group default qlen 1000
    link/ether 08:00:27:c6:ca:e0 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::a00:27ff:fec6:cae0/64 scope link
       valid_lft forever preferred_lft forever
root@debServer:~#
```

Configurez des interfaces réseaux...

Toutes les configurations de réseau sur les cartes gérées par la commande `ip` sont dynamiques. Elles seront donc perdues entre chaque "reboot" du serveur.

Pour configurer de manière statique les cartes réseaux de Linux, il est nécessaire de saisir les informations dans des fichiers de configuration qui vont être lus pendant le boot du serveur.

...sous les distributions dérivées DEBIAN



Sous Debian et ses dérivés, le répertoire de configuration des interfaces réseaux est `/etc/network` .

Le fichier **interfaces** permet de déclarer cette configuration de manière statique.

...sous les distributions dérivées REDHAT



Pour les distributions REDHAT et dérivés (notamment CentOS), le répertoire de configuration des cartes est `/etc/sysconfig/network-scripts/` . Ce répertoire contient notamment un fichier préfixé `ifcfg-` pour chaque carte reconnue par le système.

Dans la vidéo suivante, nous allons configurer de manière statique les cartes réseaux des distributions **Debian** et **CentOS**. Nous verrons :

- la syntaxe des différents fichiers de configuration de ces distributions,
- la commande `systemctl` pour prendre en compte ces configurations en redémarrant le service réseau.

Configurer les interfaces réseaux sur UBUNTU

```
jana@jana-ubuntu:~$ sudo nano /etc/netplan/01-network-manager-all.yaml
```

```
  GNU nano 6.4          /etc/netplan/01-network-manager-all.yaml
# Let NetworkManager manage all devices on
network:
  ethernets:
    enp0s3:
      addresses: [192.168.1.3/24]
      routes:
        - to: default
          via: 192.168.1.3
          on-link: True
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
      dhcp4: false
      optional: True
  version: 2
  renderer: NetworkManager
```

```
network:

    ethernets:

       [INTERFACE]:

          Dhcp4: yes/no

          Addresses: [ADRESSE_IP/NETMASK]

          Gateway: [PASSERELLE]

          Nameservers:

              Addresses: [SERVEUR_DNS_1, SERVEUR_DNS_2]
```

- Avec :
  - **[INTERFACE]** : nom de l'interface comme ens33
  - **DHCP4** : Yes or no en fonction de l'adresse IP dynamique ou statique
  - **Adresses** : Adresse IP du périphérique dans la notation de préfixe. N'utilisez pas de masque de réseau.
  - **Gateway** : Adresse IP de la passerelle pour se connecter à un réseau extérieur
  - **Nameservers** : Adresse IP des serveurs de noms DNS
- Puis pour appliquer la nouvelle configuration réseau avec **sudo** :

```
sudo netplan apply
```

Compiler et appliquer les modifications

```
jana@jana-ubuntu:~$ sudo netplan generate
jana@jana-ubuntu:~$ sudo netplan -d apply
```

Dans le cas où vous rencontrez une erreur, essayez de déboguer pour enquêter sur le problème.
Pour exécuter le débogage, utilisez la commande avec **l'option -d** :

```
sudo netplan –d apply
```